

PROCESS SYNCHRONIZATION

Process Synchronization was introduced to handle problems that arose while multiple process executions.

Process is categorized into two types on the basis of synchronization and these are given below:

- Independent Process
- Cooperative Process

Independent Processes

Two processes are said to be independent if the execution of one process does not affect the execution of another process. Example The process that does not share any shared variable, database, files, etc.

Cooperative Processes

Two processes are said to be cooperative if the execution of one process affects the execution of another process. These processes need to be synchronized so that the order of execution can be guaranteed. Example The process that share file, variable, database, etc are the Cooperating Process.

PROCESS SYNCHRONIZATION

When two or more process cooperates with each other, their order of execution must be preserved otherwise there can be conflicts in their execution and inappropriate outputs can be produced.

A cooperative process is the one which can affect the execution of other process or can be affected by the execution of other process. Such processes need to be synchronized so that their order of execution can be guaranteed.

The procedure involved in preserving the appropriate order of execution of cooperative processes is known as Process Synchronization.

It is the condition where several processes tries to access the resources and modify the shared data concurrently and outcome of the process depends on the particular order of execution that leads to data inconsistency, this condition is called **Race Condition**. This condition can be avoided using the technique called Synchronization or Process Synchronization, in which we allow only one process to enter and manipulates the shared data in Critical Section.

If two processes both make use of the same global variable and both perform reads and writes on that variable, then the order in which the various reads and writes are executed is critical.

It is difficult for the operating system to manage the allocation of resources.

It becomes very difficult to locate a programming error because results are typically not deterministic and reproducible.

Example :-

consider the following procedure;

```
void echo c
{
    chin = getch(c);
    chout = chin;
    putchar (chout);
}
```

The sharing of main memory among processes is useful to permit efficient and close interaction among processes. However, such sharing can lead to problems.

Consider the following sequence:

→ Process P_1 invokes the 'echo' procedure and is interrupted immediately after the conclusion of the 'getchar' function. At this point, most recently entered character 'x' is stored in variable 'ch1'.

→ Process P_2 is activated and invokes the 'echo' procedure, which runs to conclusion, inputting and then displaying a single character 'y'.

→ Process P_1 is resumed. By this time, the value 'x' has been overwritten in 'ch1' and therefore lost. Instead, 'ch1' contains 'y' which is transferred to 'ch2' and displayed.

Thus, the first character is lost and the second character is displayed twice. The essence of this problem is the shared global variable 'ch'. Multiple processes have access to this variable. If one process updates the global variable and then is interrupted, another process may alter the variable before the first process can use its value.

Suppose, however, that we say only one process at a time may be in that procedure. Then the foregoing sequence would result in the following:

- Process P_1 invokes the 'echo' procedure and is interrupted immediately after the conclusion of the input function. At this point, the most recently entered value 'x' is stored in 'ch'.
- Process P_2 is activated and invokes the 'echo' procedure. However, because P_1 is still inside the echo procedure, although currently suspended, P_2 is blocked from entering the procedure. Therefore, P_2 is suspended awaiting the availability of 'echo' procedure.
- At some later time, process P_1 is resumed and completes execution of 'echo'. The proper character 'x' is displayed.

INTER PROCESS COMMUNICATION

Inter process communication is the mechanism provided by the operating system that allows processes to communicate with each other. This communication could involve a process letting another process know that some event has occurred or the transferring of data from one process to another.

A diagram that illustrates inter process communication is as follows –



Synchronization in Inter process Communication

Synchronization is a necessary part of inter process communication. It is either provided by the inter process control mechanism or handled by the communicating processes. Some of the methods to provide synchronization are as follows –

- **Semaphore:** A semaphore is a variable that controls the access to a common resource by multiple processes. The two types of semaphores are binary semaphores and counting semaphores.
- **Mutual Exclusion:** Mutual exclusion requires that only one process thread can enter the critical section at a time. This is useful for synchronization and also prevents race conditions.
- **Barrier:** A barrier does not allow individual processes to proceed until all the processes reach it. Many parallel languages and collective routines impose barriers.
- **Spinlock:** This is a type of lock. The processes trying to acquire this lock wait in a loop while checking if the lock is available or not. This is known as busy waiting because the process is not doing any useful operation even though it is active.

There are several reasons for providing an environment that allows process co-operation.

- Information sharing
- Computation speed-up
- Modularity
- Convenience

Information Sharing:

Several users may be interested in same piece of information.

Eg: Shared memory

Must provide environment to allow concurrent access to these types of resources.

Computation speed-up:

- To run a particular task faster, we must break that it into subtasks.
- Each of these subtasks will execute in parallel with other.
- This is possible only if when computer has multiple processing elements (CPU or I/O channels).

Modularity:

- For co-operating between the systems, we have to construct the system in modular fashion.
- Dividing the system functions into separate processes or threads.

Convenience:

- Even an individual user may work on many tasks at the same time.
- For instance, a user may be editing, compiling and printing in parallel.

Approaches to Inter Process Communication

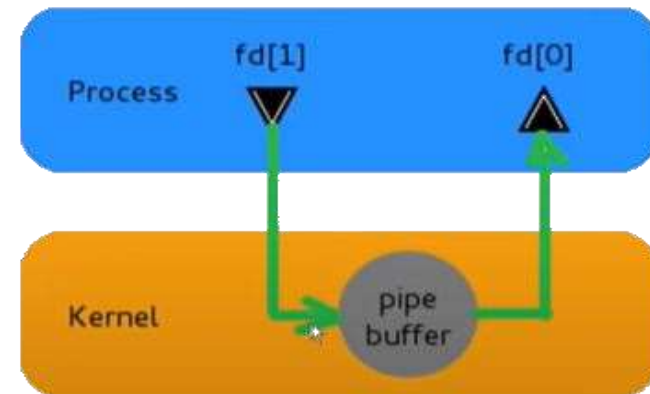
There are some different approaches to inter-process communication which are as follows:

- 1.Pipes**
- 2.Shared Memory**
- 3.Message Queue**
- 4.Direct Communication**
- 5.Indirect communication**
- 6.Message Passing**
- 7.FIFO**

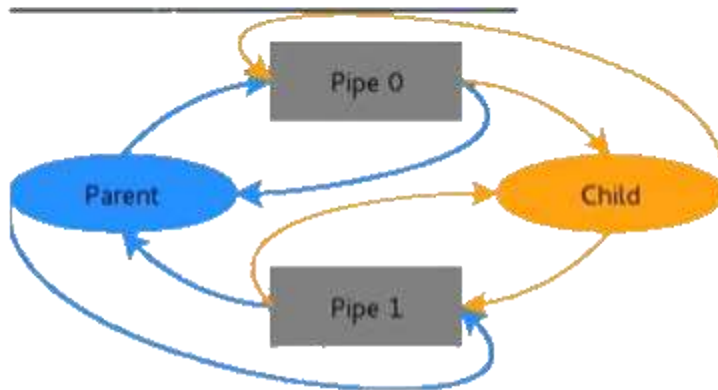
Pipe:-

The pipe is a type of data channel that is unidirectional in nature. It means that the data in this type of data channel can be moved in only a single direction at a time. Still, one can use two-channel of this type, so that he can able to send and receive data in two processes. Typically, it uses the standard methods for input and output. These pipes are used in all types of POSIX systems and in different versions of window operating systems as well.

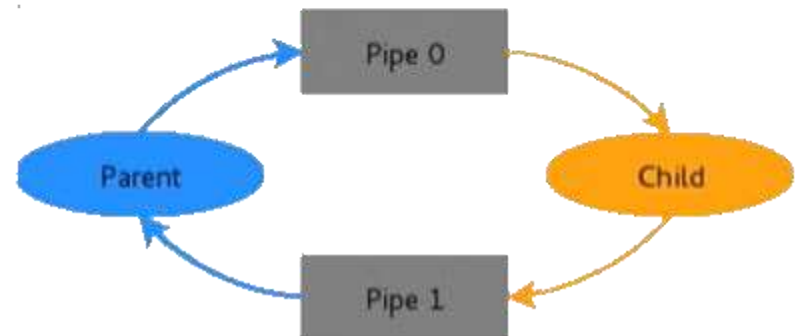
- Always **between parent and child**
- Always **unidirectional**
- Accessed by two associated file descriptors:
 - fd[0] for reading from pipe
 - fd[1] for writing to the pipe



Pipes for two way communication



- Two pipes opened
pipe0 and pipe1
- Note the unnecessary
pipes



- Close the unnecessary
pipes

Shared Memory:

In shared memory model, a region of memory that is shared by co-operating process is established.

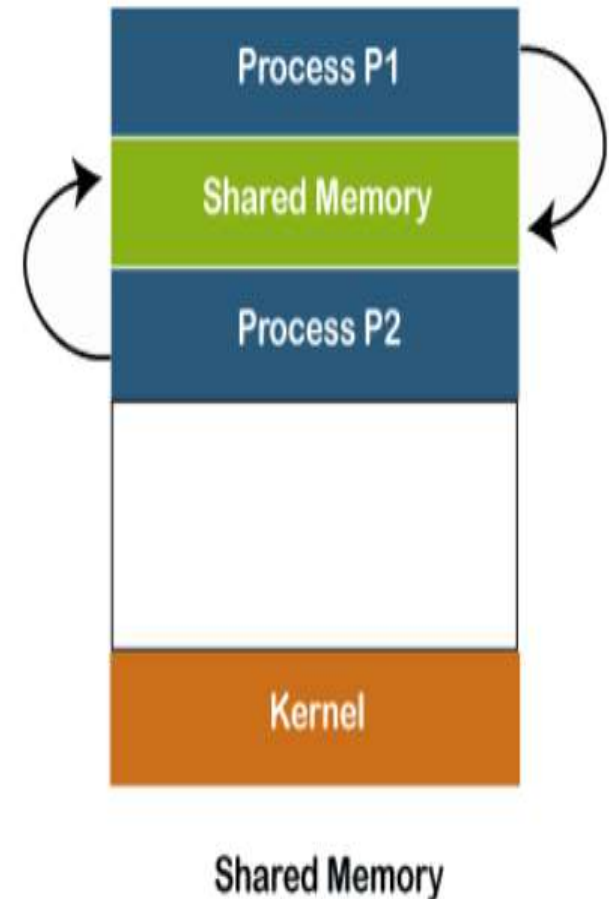
Processes then can exchange information by reading and writing data to the shared region.

Typically, a shared memory region resides in the address space of the process creating the shared memory segment.

other process that wish to communicate using the shared memory segment must attach it to their address space.

Normally, the operating system tries to prevent one process from accessing another process's memory.

Shared memory requires that two or more processes agree to remove this restriction.



Example: Producer and Consumer Problem

A **producer process** produces a information that is consumed by **consumer process**.

For Ex: A compiler may produce assembly code, which is consumed by assembler. The assembler, in turn may produce object modules, which are consumed by loader.

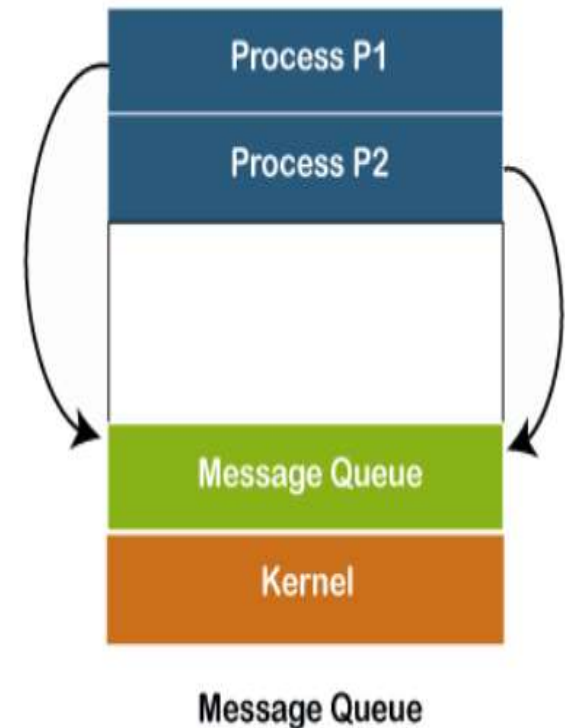
- One solution to the producer-consumer problem uses **shared memory**.
- To allow producer and consumer processes to run concurrently, we must have available a **buffer of items** that can be filled by the producer and emptied by the consumer.
- This buffer will resides in a region of memory that is shared by the producer and consumer processes.
- A producer can produce one item while the consumer is consuming another item.
- The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Message Passing:

Message passing model allows multiple processes to read and write data to the message queue without being connected to each other.

Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for inter process communication and are used by most operating systems.

It provides a mechanism to allow processes to communicate and synchronize their actions without sharing the same address space and is particularly useful in a distributed environment, where the communicating process may reside on different computers connected by a network.



A message passing facility provides atleast two operations:

- Send(message)
- Receive(message)

Message sent by a process can be of either fixed or variable size.

Fixed size:

- The system level implementation is straight forward.
- But makes the task of programming more difficult.

Variable size:

- Requires a more complex system level implementation.
- But the programming task becomes simpler.

If processes **P** and **Q** want to communicate, they must **send messages to** and **receive messages from each other**.

A **communication link** must exist between them.



This link can be implemented in a variety of ways. There are **several methods** for **logically implementing a link** and the **send() / receive()** operations, like:

- **Direct or indirect communication**
- **Synchronous or asynchronous communication**
- **Automatic or explicit buffering**

There are several issues related with features like:

- Naming
- Synchronization
- Buffering

Naming

Processes that want to communicate must have a way to refer to each other. They can use either **direct** or **indirect** communication.

Under direct communication- Each process that wants to communicate must explicitly name the recipient or sender of the communication.

- send (P, message) - Send a message to process P.
- receive (Q, message) - Receive a message from process Q.

A communication link in this scheme has the following properties:

- A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.
- A link is associated with exactly two processes.
- Between each pair of processes, there exists exactly one link.

This scheme exhibits **symmetry** in addressing;

Another variant of Direct Communication- Here, only the sender names the recipient; the recipient is not required to name the sender.

- send (P, message) - Send a message to process P.
- receive (id, message) - Receive a message from any process;
the variable id is set to the name of the process with which communication has taken place.

This scheme employs **asymmetry in addressing.**

The **disadvantage** in both of these schemes (**symmetric and asymmetric**) is the **limited modularity** of the resulting process definitions.

Changing the identifier of a process may necessitate examining all other process definitions.



With indirect communication:

The messages are sent to and received from **mailboxes**, or ports.

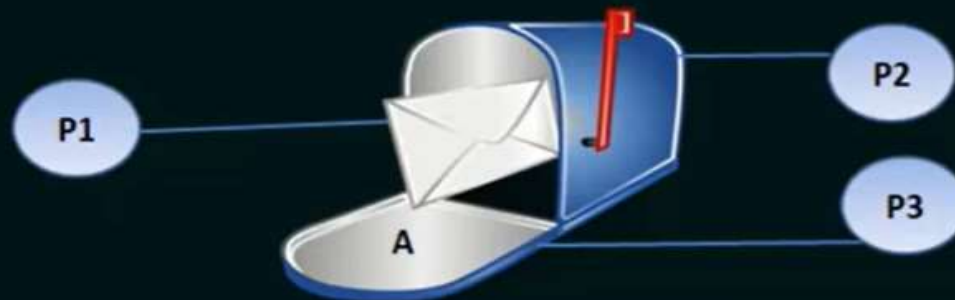


- A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.
- Each **mailbox** has a **unique identification**.
- **Two processes** can communicate only if the processes have a **shared mailbox**
 - **send (A, message)** — Send a message to mailbox A.
 - **receive (A, message)** — Receive a message from mailbox A.

A communication link in this scheme has the following properties:

- A link is established between a pair of processes only if both members of the pair have a shared mailbox.
- A link may be associated with more than two processes.
- Between each pair of communicating processes, there may be a number of different links, with each link corresponding to one mailbox.

Now suppose that processes **P1**, **P2**, and **P3** all share mailbox **A**



Process **P1** sends a message to **A**, while both **P2** and **P3** execute a receive() from **A**. **Which process will receive the message sent by P1?**

The answer depends on which of the following methods we choose:

- Allow a link to be associated with two processes at most.
- Allow at most one process at a time to execute a receive () operation.
- Allow the system to select arbitrarily which process will receive the message (that is, either P2 or P3, but not both, will receive the message). The system also may define an algorithm for selecting which process will receive the message (that is, round robin where processes take turns receiving messages). The system may identify the receiver to the sender.

A **mailbox** may be **owned** either by a **process** or by the **operating system**.

Synchronization

Communication between processes takes place through calls to **send()** and **receive ()** primitives. There are different design options for implementing each primitive.

Message passing may be either **blocking** or **nonblocking**— also known as **synchronous** and **asynchronous**.

Message passing may be either **blocking** or **nonblocking**— also known as **synchronous** and **asynchronous**.

Blocking send: The sending process is blocked until the message is received by the receiving process or by the mailbox.

Nonblocking send: The sending process sends the message and resumes operation.

Blocking receive: The receiver blocks until a message is available.

Nonblocking receive: The receiver retrieves either a valid message or a null.

Buffering

Whether communication is **direct** or **indirect**, messages exchanged by communicating processes reside in a temporary queue.

Basically, such queues can be implemented in three ways:

- Zero capacity:** The queue has a maximum length of **zero**; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.
- Bounded capacity:** The queue has finite **length n**; thus, **at most n messages can reside in it**. If the queue is not full when a new message is sent, the message is placed in the queue and the sender can continue execution without waiting. The link's capacity is finite, however. If the link is full, the sender must block until space is available in the queue.
- Unbounded capacity:** The queue's **length is potentially infinite**; thus, any number of messages can wait in it. The sender never blocks

Bounded buffer problem :-

(Producer - Consumer problem)

There are one or more producers generating some type of data (records, characters) and placing these in a buffer. There is a single consumer that is taking items out of buffer one at a time. The system is to be constrained to prevent the overlap of buffer operations. That is, only one agent (producer or consumer) may access the buffer at any one time. If buffer is full then the producer should wait and if buffer is empty then the consumer should wait. Such synchronization can be achieved through semaphore.

One solution of this problem is to use semaphores. The semaphores which will be used here are:

- **mutex**, a **binary semaphore** which is used to acquire and release the lock.
- **empty**, a **counting semaphore** whose initial value is the number of slots in the buffer, since, initially all slots are empty.
- **full**, a **counting semaphore** whose initial value is 0.

At any instant, the current value of empty represents the number of empty slots in the buffer and full represents the number of occupied slots in the buffer.

The Producer Operation

```
do:
{
    //wait until empty>0 and then decrement 'empty'
    wait(empty);
    // acquire lock
    wait(mutex);
    /* perform the insert operation in a slot */
    //release lock
    signal(mutex);
    // increment 'full'
    signal(full);
} while(TRUE);
```

- Looking at the above code for a producer, we can see that a producer first waits until there is atleast one empty slot.
- Then it decrements the **empty** semaphore because, there will now be one less empty slot, since the producer is going to insert data in one of those slots.
- Then, it acquires lock on the buffer, so that the consumer cannot access the buffer until producer completes its operation.
- After performing the insert operation, the lock is released and the value of **full** is incremented because the producer has just filled a slot in the buffer.

The Consumer Operation

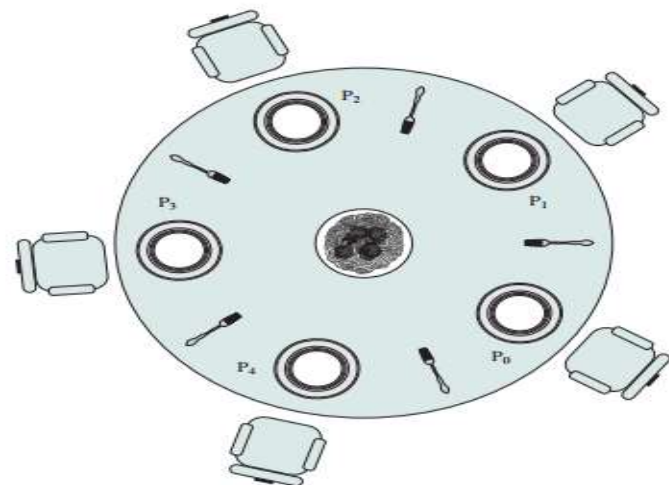
```
do:
{
    //wait until full>0 and then decrement 'full'
    wait(full);
    // acquire lock
    wait(mutex);
    /* perform the remove operation in a slot */
    //release lock
    signal(mutex);
    // increment 'empty'
    signal(empty);
} while(TRUE);
```

- The consumer waits until there is atleast one full slot in the buffer.
- Then it decrements the **full** semaphore because the number of occupied slots will be decreased by one, after the consumer completes its operation.
- After that, the consumer acquires lock on the buffer.
- Following that, the consumer completes the removal operation so that the data from one of the full slots is removed.
- Then, the consumer releases the lock.
- Finally, the **empty** semaphore is incremented by 1, because the consumer has just removed data from an occupied slot, thus making it empty.

Dining Philosophers Problem



- Let us consider five philosophers who spend their lives thinking and eating. The philosophers share a common circular table surrounded by five chairs, each belonging to one philosopher.
- In centre of table there is a bowl of rice(spaghetti) and the table is laid with five single chopsticks.
- When philosopher thinks: she does not interact with her colleagues.
- From time to time philosopher gets hungry and tries to pick up two chopsticks that are closest to her(i.e., the left and right chopsticks).
- The philosopher has to pick only one chopstick at a time(i.e., only left chopstick first). Make sure that all the philosophers are not eating at the same time.
- Philosopher should not wait for one chopstick. If two chopsticks are available then only she has to pick. Otherwise, she has to wait.



Here's the Solution

From the problem statement, it is clear that a philosopher can think for an indefinite amount of time. But when a philosopher starts eating, he has to stop at some point of time. The philosopher is in an endless cycle of thinking and eating.

An array of five semaphores, `chopstick[5]`, for each of the five chopsticks.
Initially `chopstick[i]=0`, where $i=0$ to 4;

The code for each philosopher looks like:

```
do
{
    wait(chopstick[i]);
    /* mod is used because if i==5, next
       chopstick is 1 (dining table is circular) */
    wait(chopstick[(i+1) mod 5]);
    /* eat */
    signal(chopstick[i]);
    signal(chopstick[(i+1) mod 5]);
    /* think */
}while(TRUE);
```

When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick. Then he waits for the right chopstick to be available, and then picks it too. After eating, he puts both the chopsticks down.

But if all five philosophers are hungry simultaneously, and each of them pickup one chopstick, then a deadlock situation occurs because they will be waiting for another chopstick forever.

The possible solutions for this are:

- A philosopher must be allowed to pick up the chopsticks only if both the left and right chopsticks are available.
- Allow only four philosophers to sit at the table. That way, if all the four philosophers pick up four chopsticks, there will be one chopstick left on the table. So, one philosopher can start eating and eventually, two chopsticks will be available. In this way, deadlocks can be avoided.
- An assymmetric solution, in which odd philosopher pick left chopstick first and then right. Where as, even philosopher picks right chopstick first and then left.

Deadlocks

- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

System Model

- System consists of resources
- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

Resource-Allocation Graph

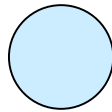
A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

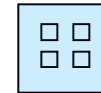
Resource-Allocation Graph

(Cont.)

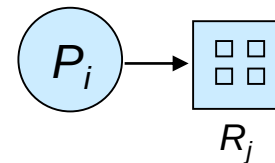
- Process



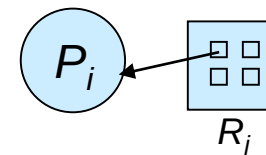
Resource Type with 4 instances



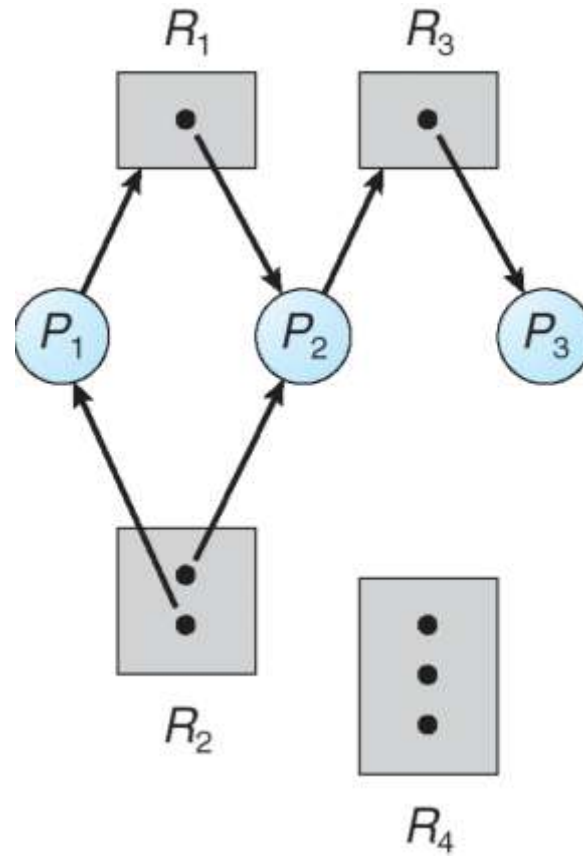
- P_i requests instance of R_j



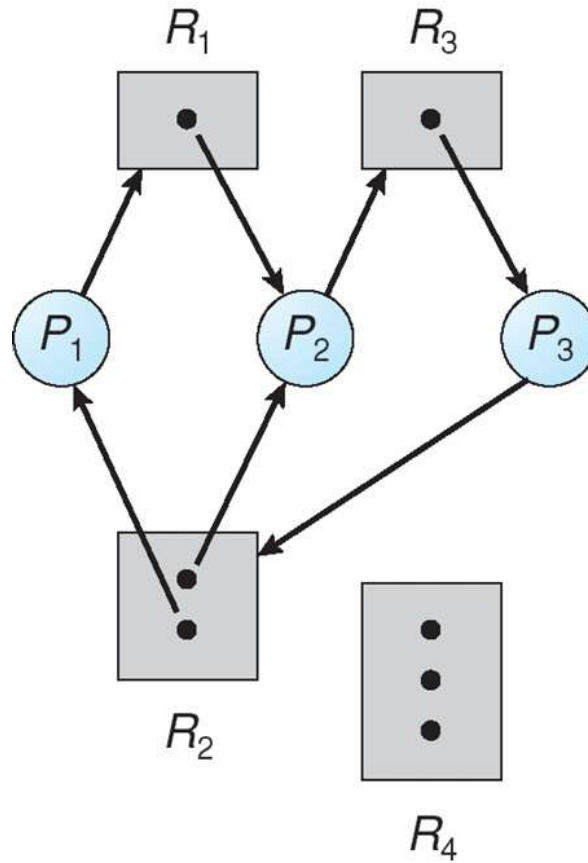
- P_i is holding an instance of R_j



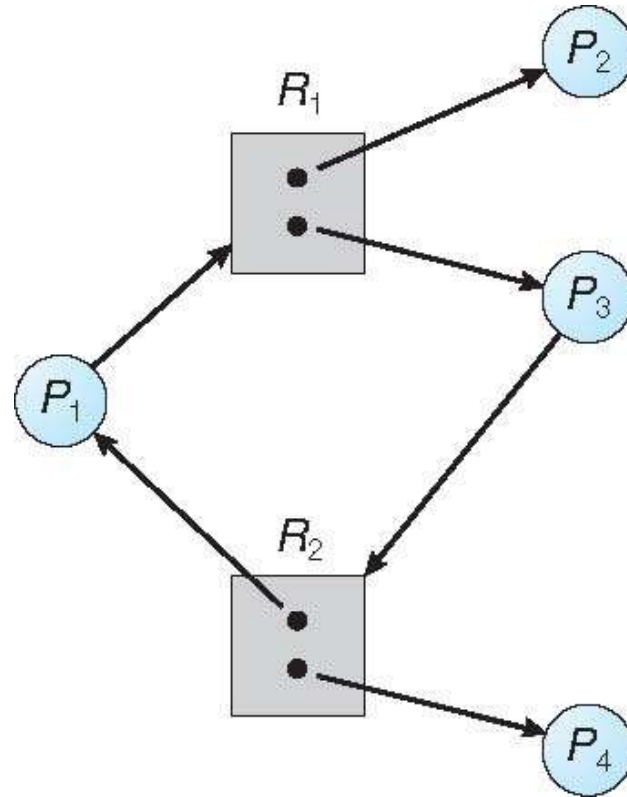
Example of a Resource Allocation Graph



Resource Allocation Graph With A Deadlock



Graph With A Cycle But No Deadlock



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock.

Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover.
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX.

Deadlock Prevention

- **Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources.
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources.
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
 - Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

- **No Preemption** –
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
 - Preempted resources are added to the list of resources for which the process is waiting.
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.
- **Circular Wait** – Impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

Deadlock Avoidance

Requires that the system has some additional *a priori* information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

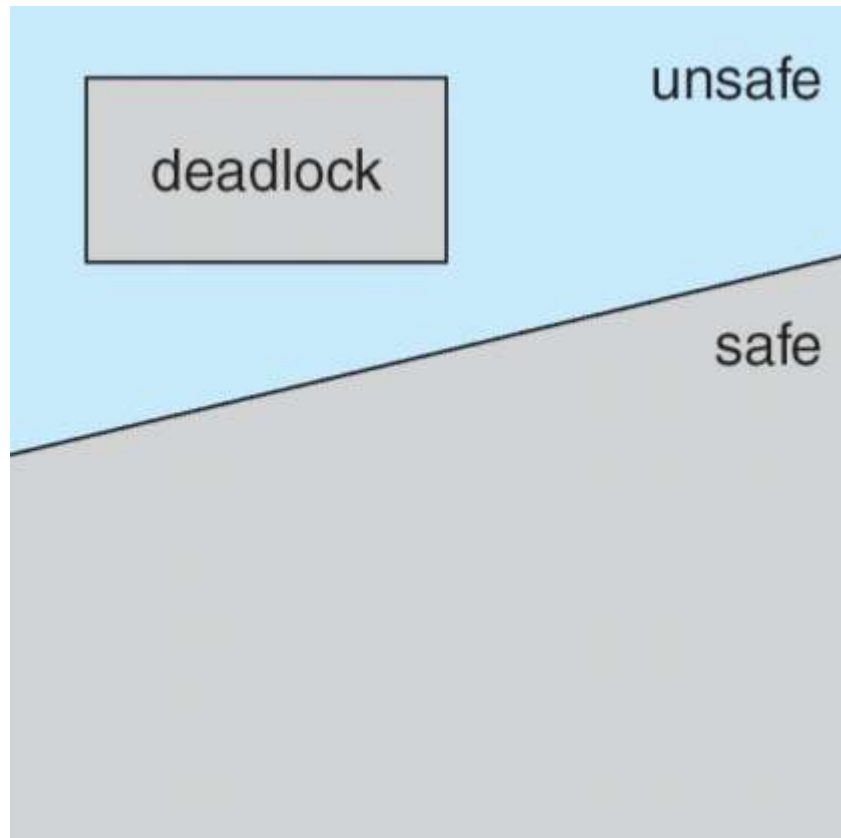
Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state.
- System is in **safe state** if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the systems such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$
- That is:
 - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
 - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



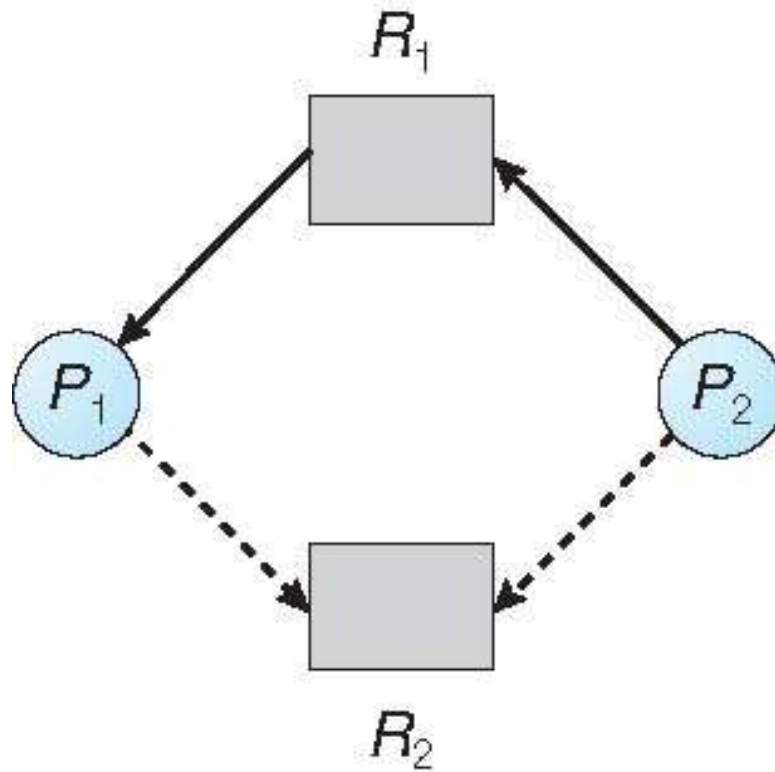
Avoidance Algorithms

- Single instance of a resource type
 - Use a **resource-allocation graph** (RAG)
- Multiple instances of a resource type
 - Use the **Banker's algorithm**

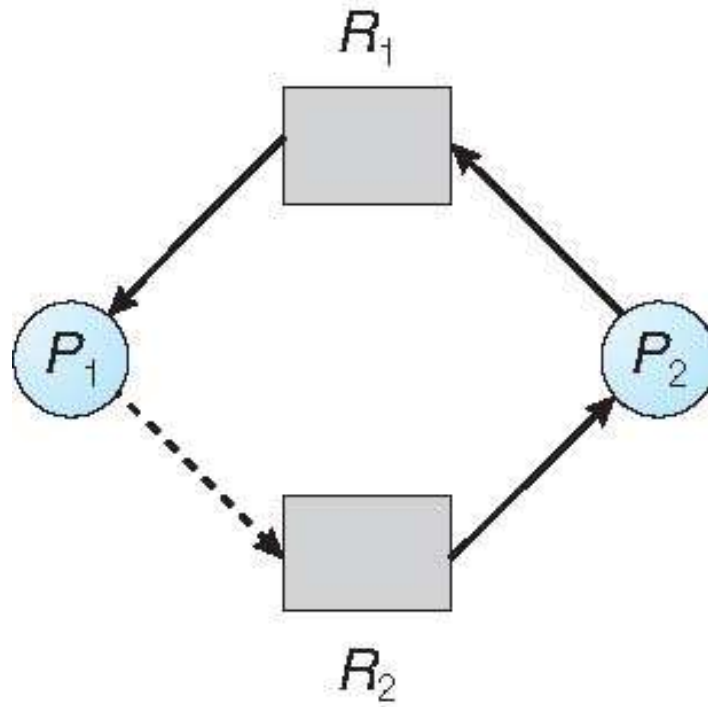
Resource-Allocation Graph Scheme

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource.
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge.
- Resources must be claimed *a priori* in the system.

Resource-Allocation Graph



Unsafe State In Resource-Allocation Graph



Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

Banker's Algorithm

- Multiple instances
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

Work = **Available**

Finish [i] = **false** for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) **Finish** [i] = **false**

(b) **Need** _{i} ≤ **Work**

If no such i exists, go to step 4

3. **Work** = **Work** + **Allocation** _{i}

Finish [i] = **true**

go to step 2

4. If **Finish** [i] == **true** for all i , then the system is in a safe state

Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If **$Request_i[j] = k$** then process P_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** , go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	<u><i>Need</i></u>
	<i>A B C</i>
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria

Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?

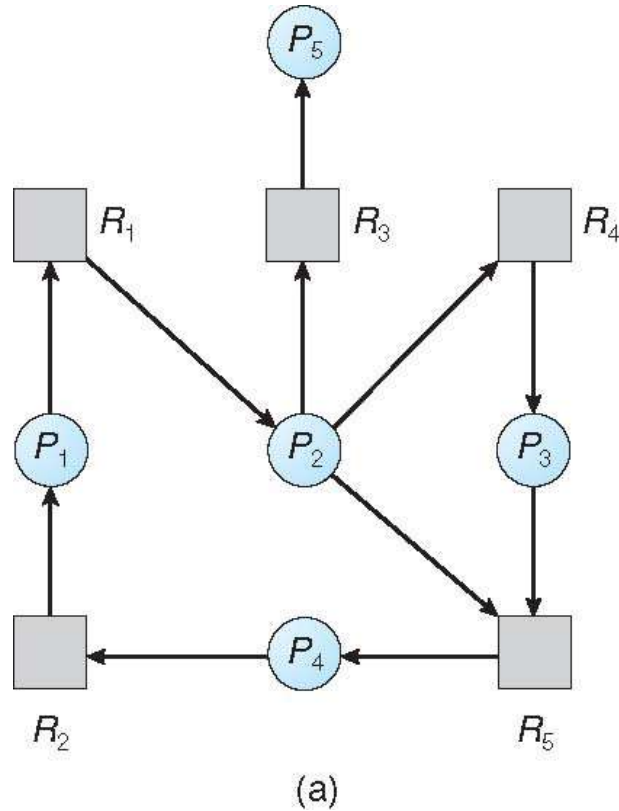
Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

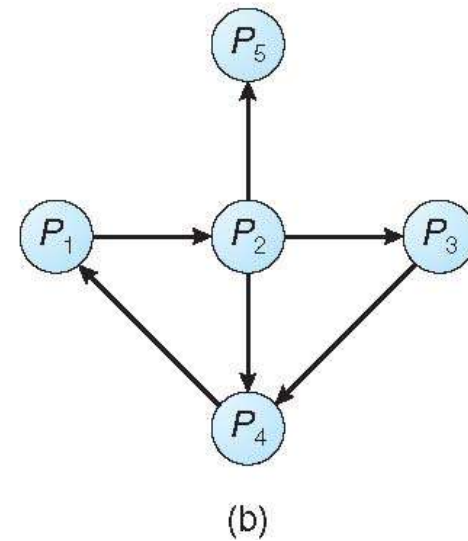
Single Instance of Each Resource Type

- Maintain **wait-for** graph
 - Nodes are processes
 - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



Corresponding wait-for graph

Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process. If **Request** $[i][j] = k$, then process P_i is requesting k more instances of resource type R_j .

Detection Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively
Initialize:

(a) ***Work = Available***

(b) For ***i = 1, 2, ..., n***, if ***Allocation_i ≠ 0***, then
Finish[i] = false; otherwise, ***Finish[i] = true***

2. Find an index ***i*** such that both:

(a) ***Finish[i] == false***

(b) ***Request_i ≤ Work***

If no such ***i*** exists, go to step 4

Detection Algorithm (Cont.)

3. **$Work = Work + Allocation_i$**
 $Finish[i] = true$
go to step 2
4. If **$Finish[i] == false$** , for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if **$Finish[i] == false$** , then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in ***Finish[i] = true*** for all i

Example (Cont.)

- P_2 requests an additional instance of type C

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

Recovery from Deadlock: Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
 1. Priority of the process
 2. How long process has computed, and how much longer to completion
 3. Resources the process has used
 4. Resources process needs to complete
 5. How many processes will need to be terminated
 6. Is process interactive or batch?

Recovery from Deadlock: Resource Preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor